Project 1 Report IN4310

Theodor Wålberg

Introduction

This exercise was intended to demonstrate an end-to-end deep learning for image analysis project. Starting with defining image data loaders, then training a ResNet neural network, and interpreting the training process. The classification task was to distinguish between six different nature and city landscapes.

Task 1

Dataset Loading

I started by splitting the dataset into three sets: training, validation, and test. The training set was used to train the model weights. The validation set was used to tune the hyperparameters when experimenting over multiple training runs. Finally, the test set was used to give a final evaluation of the model's performance on unseen data. The test set is meant to assess generalization of the model to unseen data. I used a validation set size of 2,000, a test set size of 3,000, and a training set size of 12,034.

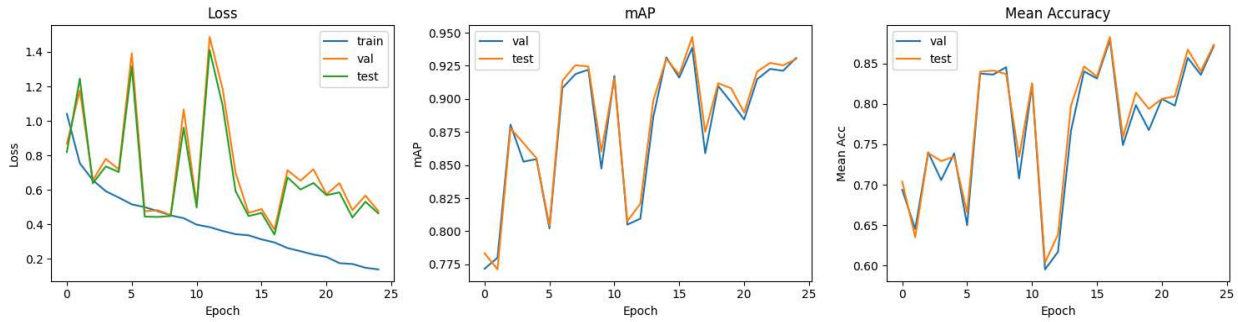
Datasets were verified to be disjoint by converting each split into a python set() and then checking if any of the sets overlap by looking for pairwise set intersections using the '&' operator. This is important in order to monitor for data leakage which could cause the validation or testing metrics to give misleading results.

Model Training

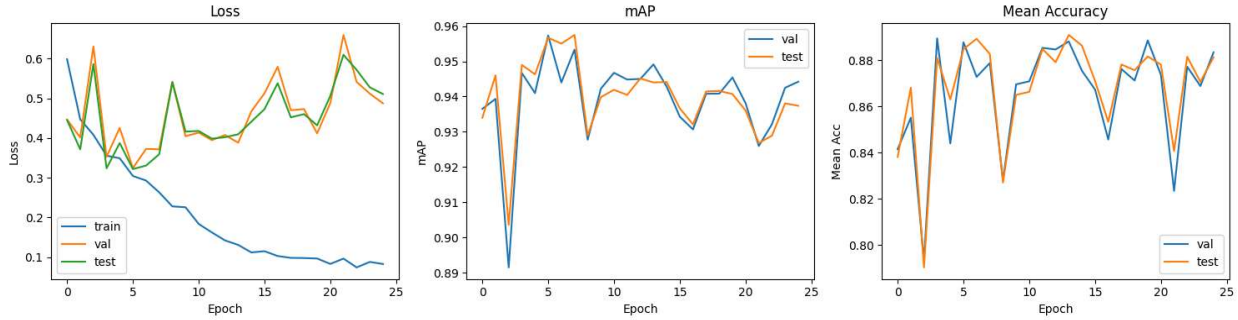
For model training I used my personal PC with an NVIDIA GeForce GTX 1650 with 4GB VRAM. To ensure reproducibility I containerized the application and allocated 2GB of VRAM as specified and mounted relevant volumes to ensure the saved models remain persistent on my local hard drive.
The metrics computed after each epoch were training loss, validation loss, test loss, accuracy, and mAP. Normally test loss would never be monitored during training since it can bias the hyperparameter tuning procedure in a way that leads to overfitting on the test set. This was done purely for demonstration purposes in this exercise.

Three initial models were trained. All models used the ResNet-18 model architecture and were completely trained from scratch over 25 epochs with a batch size of 16. No transfer learning was used, only randomly initialized weights with a fixed seed for reproducibility. Each model's hyperparameters are defined in a configuration file. Model 1 had a learning rate of 0.001, used the Adam optimizer with 0.0001 weight decay and no augmentation. Model 2 had a learning rate of 0.01, used the SGD optimizer with 0.0005 momentum, and no augmentation. Model 3, like Model 1, used a learning rate 0.001, Adam optimizer, and 0.0001 weight decay, but this time with augmentation. The augmentation for model 3 worked as follows: For each time the trainer sampled from the train split via the data loader, a set of random transforms were added. I used the torchvision transforms library which allows for composing multiple transforms into one operation then applying that transform to an image tensor. I used random cropping and random horizontal flips. This was chosen because it is realistic that a scene might be cropped by pointing a camera in a different direction while zooming in. Horizontal flips were chosen instead of vertical since it is unlikely that someone takes an upside-down photo of a landscape. It is important to note that my current augmentation scheme does not produce more training samples per epoch. I could have performed N augmentations on the same image and produced N x 12,034 training samples, but I did not, and it was intentional. The reason for not extending my dataset was that it is already a relatively large dataset and training time was limited by the project duration. The current augmentation scheme still acts as a good regularizer by introducing random variations each time a training image is sampled. This effectively means the model has a harder time overfitting / memorizing the training set.
During training, the final model weights were updated after every epoch whenever the validation metrics showed the best mAP for that training run.
Then I trained the same three models but instead of randomly initializing the weights, I downloaded the pretrained ResNet-18 weights from torchvision.models and then fine-tuned the weights using our dataset. This required replacing the model head layer to accommodate for the new number of classes in our case.

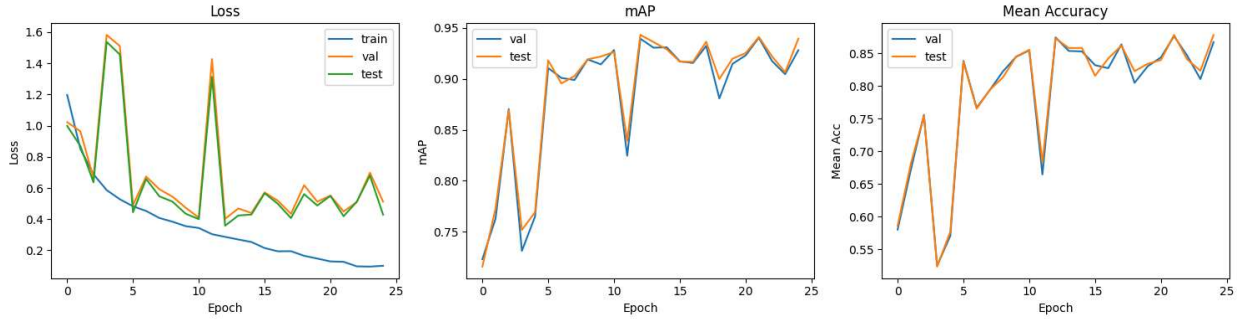
Model 1:



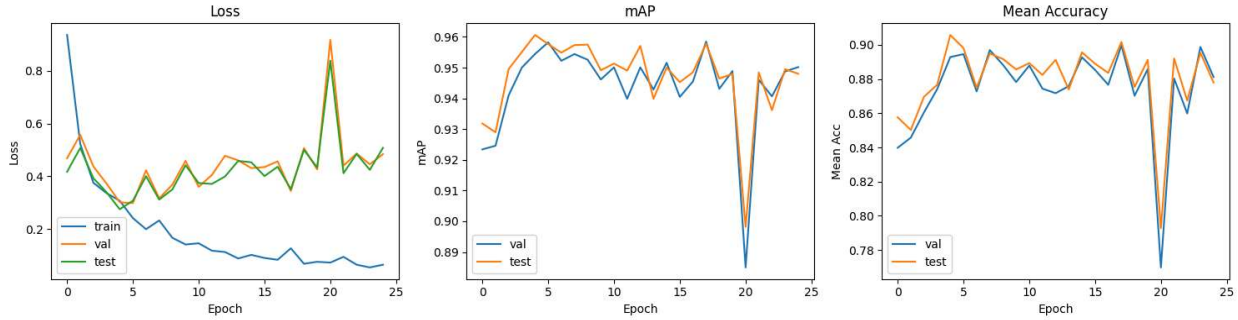
Model 1 transfer:



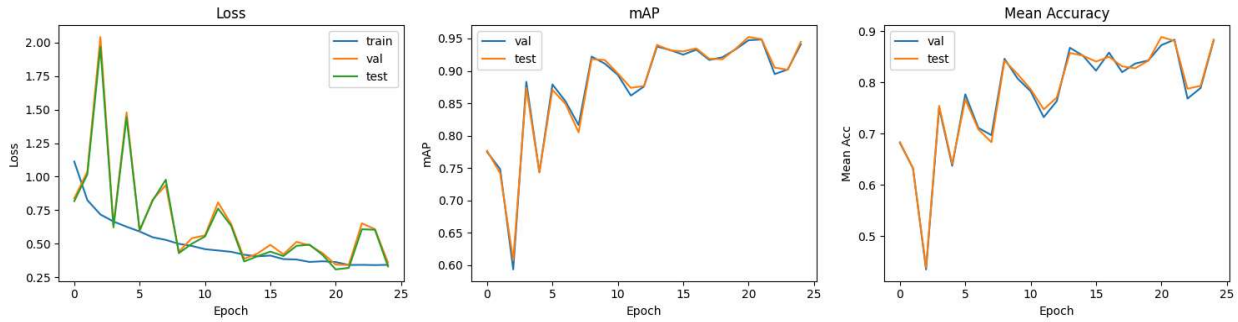
Model 2:



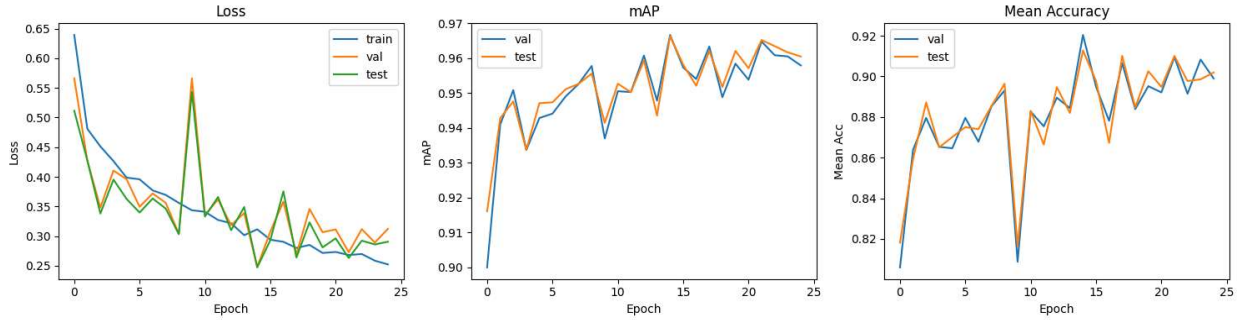
Model 2 transfer:



Model 3:



Model 3 transfer:



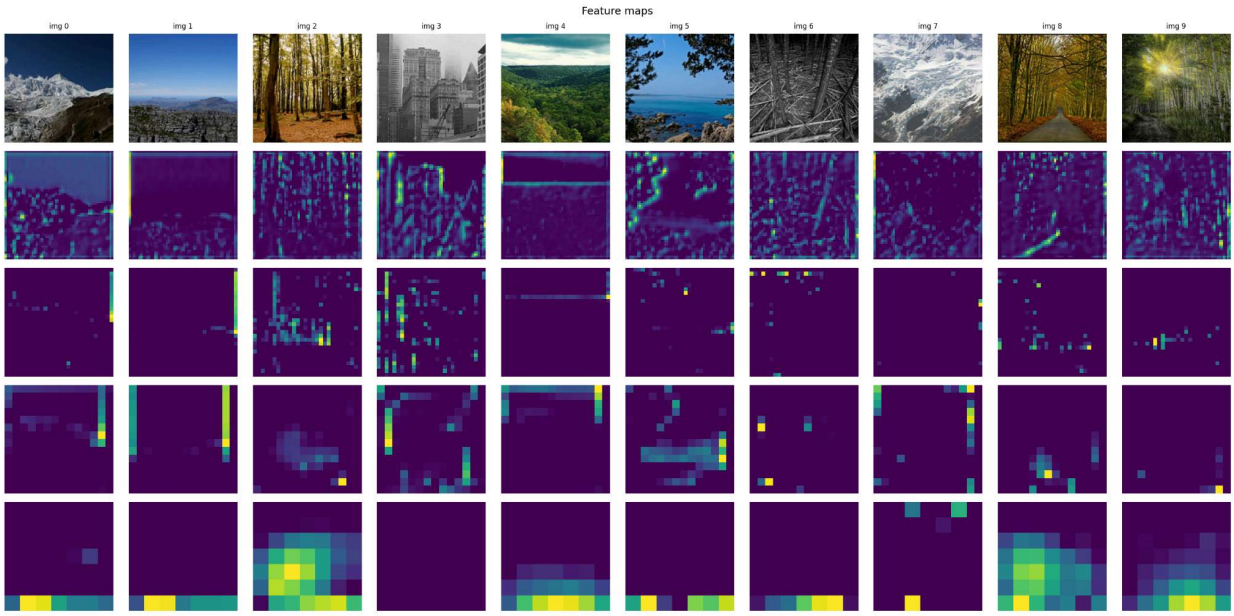
Evaluation metrics

model	split	loss	mAP	mean_acc
model1	val	0.3697	0.9386	0.8794
model1	test	0.3405	0.9468	0.8827
model2	val	0.4494	0.9403	0.8762
model2	test	0.419	0.941	0.8782
model3	val	0.3421	0.9485	0.8836
model3	test	0.3206	0.9489	0.8813
model1_transfer	val	0.3245	0.9573	0.8878
model1_transfer	test	0.322	0.9567	0.8848
model2_transfer	val	0.344	0.9585	0.8997
model2_transfer	test	0.351	0.9578	0.9017
model3_transfer	val	0.2481	0.9667	0.9204
model3_transfer	test	0.247	0.9664	0.9127

Task 2

Feature Map Statistics

Here I used PyTorch's forward hooks to visualize activations from selected layers of model 3.



layers: ['layer1', 'layer2', 'layer3', 'layer4']
generating feature maps
five modules: ['conv1', 'layer1', 'layer2', 'layer3', 'layer4']

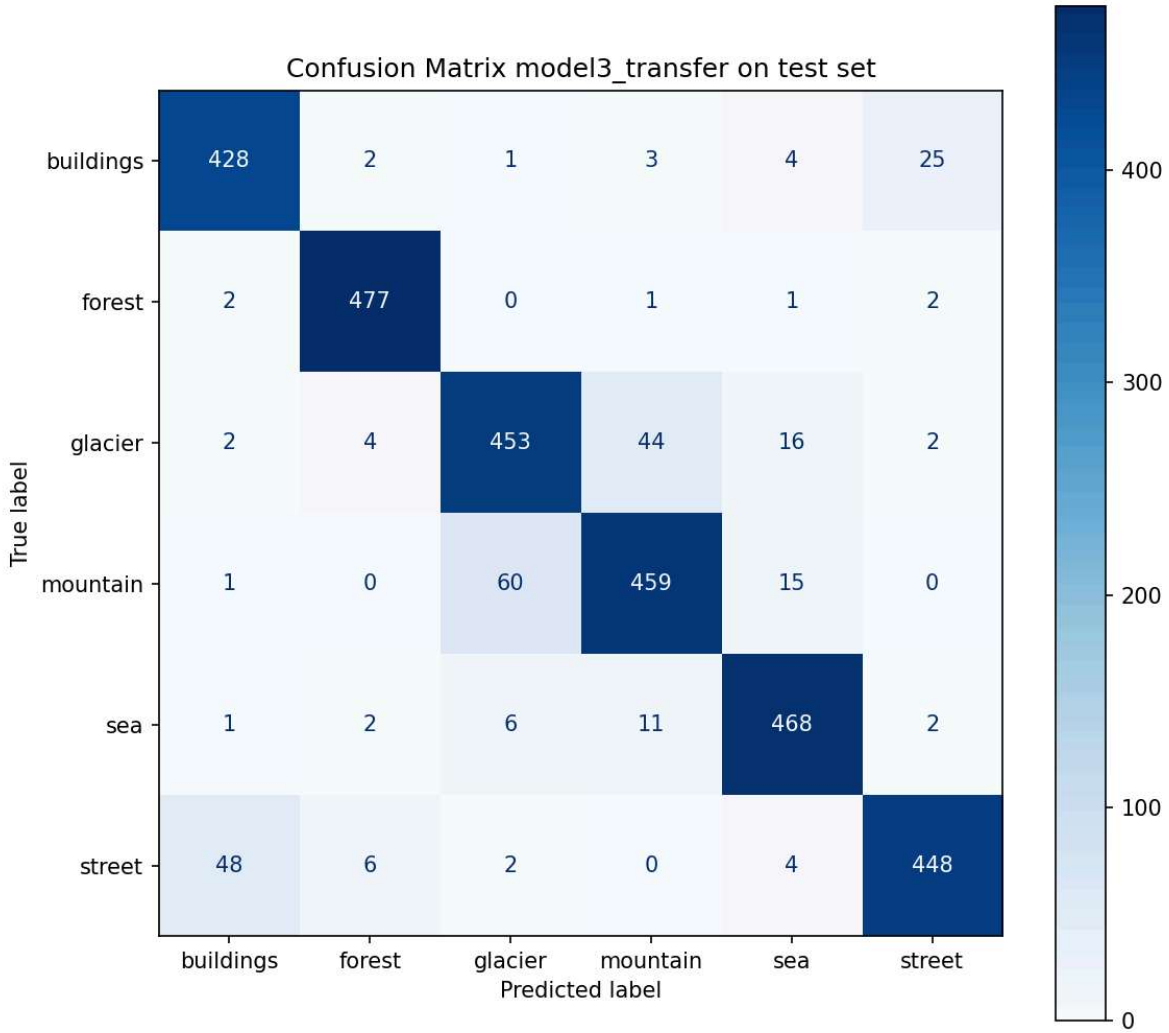
The earlier layers activate in a way that visually aligns with lower level features, such as edges, corners, etc. While the later layers seem to activate in broader abstract regions of interest. This corresponds with how the model is designed to work.

Sparsity of the feature maps by layer:

Layer	module avg % non-positive
conv1	52.76%
stage1	33.59%
stage2	67.11%
stage3	69.53%
stage4	78.49%

Appendix

Confusion matrix



Average precision per class

model	AP_buildings	AP_forest	AP_glacier	AP_mountain	AP_sea	AP_street
model1	0.9381	0.9926	0.9218	0.8805	0.9623	0.9365
model1	0.9457	0.9978	0.9097	0.9115	0.9668	0.9489
model2	0.9379	0.9898	0.9104	0.8922	0.958	0.9534
model2	0.9452	0.9969	0.8925	0.8956	0.967	0.9487
model3	0.9619	0.993	0.9223	0.8932	0.9613	0.9591
model3	0.952	0.9983	0.9057	0.9154	0.9618	0.96
model1_transfer	0.9603	0.9974	0.9394	0.919	0.9729	0.9551
model1_transfer	0.9598	0.9988	0.922	0.9278	0.9803	0.9515
model2_transfer	0.9572	0.9976	0.9276	0.9171	0.9837	0.9676
model2_transfer	0.9616	0.9983	0.9151	0.9201	0.9807	0.9709
model3_transfer	0.9746	0.9968	0.9468	0.9383	0.9736	0.9701
model3_transfer	0.9711	0.9989	0.9317	0.9367	0.9859	0.9739

Accuracy per class

model	Acc_buildings	Acc_forest	Acc_glacier	Acc_mountain	Acc_sea	Acc_street
model1	0.9094	0.9689	0.8991	0.7927	0.8807	0.8254
model1	0.9179	0.9834	0.8983	0.7682	0.8878	0.8406
model2	0.8252	0.9752	0.7464	0.8852	0.9113	0.9142
model2	0.8402	0.9814	0.7505	0.8673	0.9204	0.9094
model3	0.8673	0.9627	0.7061	0.916	0.9235	0.926
model3	0.8553	0.9855	0.737	0.9103	0.902	0.8976
model1_transfer	0.7605	0.9752	0.8415	0.8459	0.9419	0.9615
model1_transfer	0.7559	0.9834	0.8349	0.8206	0.9612	0.9528
model2_transfer	0.9288	0.9783	0.8761	0.8627	0.9358	0.8166
model2_transfer	0.9352	0.9793	0.856	0.8505	0.9327	0.8563
model3_transfer	0.9385	0.9783	0.8818	0.8824	0.9388	0.9024
model3_transfer	0.9244	0.9876	0.8695	0.8579	0.9551	0.8819